

01

[§2]

Why Flask and not Django or FastAPI?

Django ships an ORM and an admin we'd have to fight — our 23-table schema is custom and the admin would hide our data-engineering work. FastAPI is superb for pure APIs but we need server-rendered pages, sessions, and templates for a public portal — a SPA would double the work.

Hook: "We picked the tool that disappears, not the one that impresses."

Card 1 of 15

02

[§2]

Why PostgreSQL and not MongoDB?

Our data is intensely relational — bins -> telemetry -> incidents -> work orders -> complaints — with foreign keys everywhere. MongoDB's schema flexibility buys us nothing here, and Postgres adds JSONB, window functions for analytics, and row-level-security policies we actually ship. Tests run on embedded Postgres so dev equals production — no "works in dev, breaks in prod."

Card 2 of 15

03

[§6A]

How many people can register with the same phone or email?

Exactly one — enforced at three layers: a friendly pre-check in the form, an IntegrityError race guard so two simultaneous signups can't both win, and database unique indexes `uq\_user\_phone` and `uq\_user\_email` as the final law. Hook: "Even raw SQL can't create a duplicate account." NULLs stay legal for phone-OTP registrations and informal pickers.

04

[§6A]

Does your site accept only garbage images? How do you know a photo is real?

Three guarantees: every upload must decode as a real image with Pillow — fail-closed, so an .exe named .jpg never lands; the photo's EXIF GPS is cross-checked against the phone's live GPS within 100 metres, so screenshots and internet images are rejected; and everything is re-encoded to strip metadata. And the AI layer is real, not a promise: a fine-tuned MobileNet (97.3% validation accuracy on 4,000 waste vs. everyday photos) ships as a 6 MB ONNX model and refuses non-waste photos with the score in the message — failing open to the decodability gate if the model ever errors.

05

[§4]

Which ML models, and why those?

Three models, each sized to its data: two scikit-learn forests on telemetry — ward miss-risk from day-of-week, season, live weather and recent complaints; fill-rate from per-ping history — plus a MobileNetV3 fine-tuned on 4,000 waste vs. everyday photos (97.3% validation) that verifies every citizen upload in-process. Forests train in seconds on a 512 MB container and degrade to transparent heuristics; the ONNX photo model fails open. Everything is reproducible from the repo — ``python train_model.py`` rebuilds the forests, ``scripts/train_photo_classifier.py`` retrains the photo model — and all three artifacts ship inside the Docker image.

Hook: "We sized the model to the data — and you can rebuild every one of them from the repo."

Card 5 of 15

06

[§4]

Why not deep learning / LSTM?

No long history — no Panchayat has last year's per-bin data — and a 512 MB free-tier container can't retrain a network. The decisions we make ("empty bin X first") need explainability a worker can argue with. Deep learning on 600 rows would overfit and pretend to know more than it does.

Card 6 of 15

07

[§3]

Is your data real?

Schedules, wards, and contacts are real field-observation data. Telemetry on the demo is seeded simulation driven through the real ingestion path — the same HMAC-signed endpoint a physical ESP32 would call. We never claim production sensors; we claim a production-real pipeline, and we say the synthetic 600-row history is a documented limitation.

08

[§5]

How much traffic can it handle?

Two gunicorn gevent workers multiplex I/O-bound requests — roughly 100–160 concurrent, on the order of 150–300k requests/day for this mix. First bottleneck is the DB pool, so hot reads are cached; telemetry writes are tiny signed POSTs; rate limiting protects auth endpoints specifically. Scaling to 2 lakh residents needs config, not a rewrite: stateless containers, Redis queue, read replicas.

09

[§6]

Walk me through the security.

Defence-in-depth for a civic system holding citizen phone numbers: OTPs are 6-digit, SHA-256 hashed at rest, 5-minute TTL, brute-force counted into account lockout; IoT devices must present HMAC-SHA256 signatures or per-device keys; complaint tracking uses signed tokens so IDs can't be enumerated; CSRF bucketed tokens on every form; Talisman headers — all six verified present today; sessions regenerate at login; every privileged action is audit-logged.

10

[§0, §2]

What happens when the OTP never arrives?

The MFA page has an explicit "Email me the code" button — staff logins work even with zero Twilio/WhatsApp credit. The default chain is WhatsApp -> SMS -> email fallback; on localhost the OTP flashes on screen for demos.

Hook: "The staff login path has no single point of failure."

11

[§6A connectivity]

What if Redis or the network dies?

Everything degrades gracefully by design: no Redis -> jobs run inline in-process; no WhatsApp keys -> email path; no Cloudinary/Supabase -> disk storage with loud admin alerts rather than silent loss; no network at all -> the service worker serves the PWA shell and complaints queue in IndexedDB, syncing automatically on reconnect.

12

[§2, §7]

How do you know the system actually works?

359 automated tests across 8 suites, running against a real embedded PostgreSQL — not SQLite mocks — so migration and RLS bugs surface locally. Plus a live `/health` endpoint that checks database, mail path, storage, and the job queue — if Redis is configured and no worker is consuming, `/health` says so out loud instead of flashing green while OTP mail silently dies — and the same suite gates every branch push in GitHub Actions. Today we also ran a 21-point end-to-end verification of every role and API on the live stack — all green.

13

[§7]

What was YOUR part in the team?

Batch of four (24331A4441/4434/4446/4426), full-stack shared. The artefact trail speaks: 23 tables, 29 Alembic migrations, 131 routes in 9 blueprints, 359 tests, 3 retrainable ML artifacts — every line reviewable on GitHub, deployed live at smartgarbage.onrender.com today.

14

[§7]

What's the cost to run?

Zero rupees today: Render free web service, Supabase free Postgres, Cloudflare free edge, WhatsApp service-window messages free, Redis optional with an in-process fallback. The first paid step only arrives if we scale beyond this ward cluster.

15

[§7]

What would you build next?

In order: real ESP32 firmware on physical bins (the OTA channel already ships in the admin console), nightly route optimisation blending the ETA predictions into worker routes, and a ward-level analytics API for the district office. The honest close: "The portal is the data producer; the ML layer is the data consumer — this project is a complete data-engineering loop, not just a CRUD site."

15-second lightning round

- Payments:

Razorpay UPI, webhook signature-verified, receipts off the request path. §2

- Bilingual:

921 strings, English + Telugu, one-click switcher. §1

- Offline:

Service worker PWA — installs without a store, queues complaints in IndexedDB. §1

- Biggest challenge:

Making the free tier reliable: queue starvation, rate-limit multiplication across gevent workers — all measured and documented. §7

- Data deletion/privacy:

Self-service deletion requests + admin workflow; phones are the only PII; EXIF stripped on upload. §6A